Bộ Câu Hỏi Bảo Vệ Đồ Án Tốt Nghiệp: Sentinel NetLab

(Sắp xếp từ dễ đến khó theo từng phần, cung cấp câu hỏi nền tảng và câu hỏi chuyên sâu)

Phần 1: Tổng quan dự án & Kiến trúc hệ thống

Câu 1: Sentinel NetLab là gì? Trả lời: Là một hệ thống phát hiện xâm nhập mạng không dây (WIDS) phân tán, lai ghép (hybrid). Nó kết hợp giữa phát hiện dựa trên luật (signature-based) và phân tích bất thường bằng Machine Learning để giám sát, cảnh báo các mối đe dọa trên mạng WiFi.

Câu 2: Tại sao gọi dự án này là hệ thống "WIDS" thay vì "WIPS"? Trả lời: Vì hệ thống chỉ tập trung vào việc giám sát, thu thập dữ liệu thụ động (Passive Monitoring) và cảnh báo (Intrusion Detection - IDS) các cuộc tấn công. Nó không thực hiện các hành động can thiệp chủ động (như gửi gói tin ngắt kết nối lại kẻ tấn công) để ngăn chặn (Intrusion Prevention - IPS), nhằm đảm bảo tuân thủ pháp luật và đạo đức bảo mật.

Câu 3: (Nền tảng) WIDS phân tán (Distributed WIDS) khác gì với WIDS tập trung? Tại sao dự án này chọn mô hình phân tán? Trả lời:

- **WIDS tập trung** dùng một cảm biến duy nhất bao phủ một khu vực nhỏ.
- **WIDS phân tán (Distributed)** sử dụng nhiều cảm biến nhỏ gọn (Sensor/Raspberry Pi) rải rác khắp tòa nhà. Dữ liệu từ các cảm biến sẽ hội tụ về một máy chủ trung tâm (Controller) để xử lý.
- **Lý do chọn:** Mạng WiFi có giới hạn vật lý về độ phủ sóng. Để giám sát một doanh nghiệp lớn, bắt buộc phải dùng nhiều cảm biến đặt ở nhiều vị trí để bao phủ toàn bộ không gian RF, đồng thời mô hình phân tán cũng giúp thực hiện được thuật toán định vị (Geo-Location Trilateration) dựa trên nhiều nguồn thu.

Câu 4: Kiến trúc của Sentinel NetLab gồm những thành phần chính nào? Trả lời: Hệ thống có 3 thành phần chính:

1. **Sensor Layer (Edge):** Các cảm biến (như Raspberry Pi) thu thập gói tin WiFi, tiền xử lý và chạy các thuật toán phát hiện tại biên.
2. **Controller Layer (Core):** Máy chủ trung tâm (Flask API) nhận dữ liệu từ cảm biến, lưu trữ, đánh giá rủi ro tổng thể và quản lý cảnh báo.
3. **Dashboard:** Giao diện web (Dash/Plotly) để trực quan hóa dữ liệu và bản đồ nhiệt theo thời gian thực.

Câu 5: Tại sao hệ thống lại xử lý thuật toán phát hiện (như Evil Twin, DoS) ngay tại Sensor thay vì gửi toàn bộ gói tin về Controller xử lý? Trả lời: Việc này gọi là Edge Computing (xử lý tại biên). Giúp giảm tải băng thông mạng (không cần gửi toàn bộ gói tin PCAP lớn về server), giảm độ trễ cảnh báo (phát hiện và cảnh báo ngay lập tức), và tăng tính riêng tư (chỉ gửi metadata/chỉ số thay vì nội dung gói tin).

Câu 6: (Khó) Trong mô hình phân tán, làm sao hệ thống giải quyết vấn đề "Trùng lặp sự kiện" (Event Deduplication) khi một cuộc tấn công WiFi bị ghi nhận đồng thời bởi nhiều cảm biến khác nhau? Trả lời: Tại trung tâm (Controller), hệ thống sử dụng module `Alerts/Dedup` (có thể ứng dụng Redis hoặc cấu trúc dữ liệu in-memory của AlertManager). Nếu nhiều cảm biến (Sensor-01, Sensor-02) cùng gửi về một cảnh báo giống hệt nhau (ví dụ Deauth Flood từ MAC A đến MAC B) trong một "khoảng thời gian cửa sổ" (`dedup_window` ví dụ 60 giây), Controller sẽ gộp (merge)

chúng lại thành một sự kiện duy nhất nhưng ghi nhận "Danh sách các cảm biến" đã phát hiện (để ứng dụng vào Geo-mapping) thay vì gửi 2 email cảnh báo riêng biệt cho người quản trị.

Câu 7: (Khó) Làm sao Controller biết một Sensor đã "chết" (Offline) hay chỉ đơn thuần là đang nằm trong vùng không có sóng (Quiet RF Environment)? Trả lời: Hệ thống sử dụng cơ chế **Heartbeat**. Cứ mỗi phút, Sensor sẽ chủ động gửi một gói tin Heartbeat (`/api/v1/heartbeat`) lên Controller kèm theo trạng thái và số lượng frame đã bắt được. Nếu Controller không nhận được Heartbeat trong một khoảng thời gian (ví dụ 3 phút), nó sẽ đánh dấu Sensor đó là Offline, bất kể sóng WiFi ở xung quanh Sensor đó có im ắng hay không.

Câu 8: (Bổ sung) "Wardriving" trong dự án đóng vai trò gì trong kiến trúc tổng thể? Trả lời: Wardriving được dùng trong "Assessment Mode" (Chế độ đánh giá) để dò quét chủ động các mạng WiFi xung quanh khu vực vật lý. Dữ liệu bao gồm BSSID, SSID, RSSI, Channel và Chuẩn bảo mật (Security). Nó giúp người quản trị lập bản đồ mạng hiện có (Baseline) từ trước, giúp phân biệt các mạng hợp pháp với các điểm phát sóng trái phép (Rogue AP) xuất hiện sau này.

Câu 9: (Mới - Kiến trúc) Nếu khối lượng dữ liệu (telemetry) từ hàng trăm cảm biến đẩy về cùng lúc, Controller (Core Backend) của dự án sẽ thiết kế/xử lý như thế nào để không bị nghẽn? Trả lời: Controller được thiết kế theo hướng bất đồng bộ (Asynchronous processing). Các payload từ Sensor gửi đến REST API sẽ được tiếp nhận nhanh và ném thẳng vào Message Broker/Queue (như Redis). Các Background Workers (như Celery) sẽ âm thầm lấy dữ liệu từ hàng đợi ra để phân tích, ghi vào Database. Nhờ vậy, API Server không bao giờ bị block/nghẽn khi xử lý I/O nặng.

Phần 2: Cảm biến (Sensor) & Lý thuyết mạng 802.11

Câu 10: (Nền tảng) Cảm biến (Sensor) hoạt động ở chế độ mạng nào để thu thập dữ liệu? Nó khác gì với chế độ Promiscuous? Trả lời: Cảm biến bắt buộc phải hoạt động ở **Monitor Mode** (chế độ giám sát).

- **Promiscuous Mode:** Bắt mọi gói tin của mạng mà thiết bị *đã kết nối* (thường dùng trong mạng dây LAN hoặc WiFi đã có mật khẩu).
- **Monitor Mode:** Bắt trực tiếp tín hiệu radio thô ở tầng vật lý (MAC Layer 802.11) của *mọi mạng* đang phát trên kênh đó mà *không cần* kết nối vào bất kỳ mạng nào.

Câu 11: (Nền tảng) Management Frames (Khung quản lý 802.11) là gì? Kể tên một vài loại frame mà dự án của em tập trung phân tích? Trả lời: Khung quản lý là các gói tin dùng để thiết lập, duy trì và kết thúc kết nối WiFi (không chứa dữ liệu người dùng). Dự án thường tập trung vào:

- **Beacon Frame:** AP phát ra định kỳ để thông báo sự tồn tại (dùng phát hiện Evil Twin, Beacon Flood).
- **Probe Request/Response:** Client dò tìm mạng (dùng trong Wardriving/Karma).
- **Deauthentication/Disassociation:** Ngắt kết nối (dùng phát hiện DoS/Deauth Flood).

Câu 12: Quá trình (Pipeline) xử lý dữ liệu của Sensor diễn ra như thế nào? Trả lời: Gồm 4 bước:

1. **Ingestor:** Driver (IwCapture/MockCapture) đọc gói tin thô.
2. **Parser & Normalizer:** Giải mã khung 802.11, chuẩn hóa dữ liệu thành JSON và ẩn danh MAC address.
3. **Analyzer:** Đưa siêu dữ liệu qua các engine phát hiện (Evil Twin, DoS, KRACK...).
4. **Exporter:** Đẩy dữ liệu vào hàng đợi (Spool queue/Backlog) để gửi lên Controller theo từng lô (batch).

Câu 13: Tính năng "Channel Hopping" trong Sensor có tác dụng gì? Trả lời: Mạng WiFi hoạt động trên nhiều kênh (channel) khác nhau. `ChannelHopper` giúp card mạng liên tục chuyển đổi giữa các kênh (với độ trễ `dwell_time` cố định) để có thể giám sát toàn bộ phổ tần thay vì chỉ mù quáng nghe trên một kênh duy nhất.

Câu 14: (Mới - Khó) Nếu dwell_time trong quá trình Channel Hopping được cấu hình quá nhanh (ví dụ 10ms) hoặc quá chậm (ví dụ 1000ms), điều gì sẽ xảy ra? Trả lời:

- Dwell time **quá nhanh**: Sensor chuyển kênh liên tục, dẫn đến việc thu thập không trọn vẹn gói tin (packet fragment) hoặc bỏ lỡ các gói beacon có chu kỳ chậm (thường là 100ms), gây mù một phần dữ liệu.
- Dwell time **quá chậm**: Sensor "lắng nghe" quá lâu ở một kênh, khiến những kênh khác bị bỏ ngỏ trong thời gian dài. Kẻ tấn công có thể chớp thời cơ tung Deauth Flood ở kênh khác mà Sensor không kịp phát hiện. Dwell time chuẩn thường từ 150ms-250ms.

Câu 15: (Khó) Làm thế nào hệ thống đảm bảo không bị quá tải bộ nhớ RAM khi phải xử lý hàng nghìn gói tin mỗi giây? Trả lời:

1. Sử dụng thiết kế Producer-Consumer: Luồng bắt gói tin (Capture) đưa vào một bộ đệm vòng (BufferManager) có giới hạn kích thước (`max_memory_items` ví dụ 10,000).
2. Luồng đẩy dữ liệu (Upload Thread) liên tục đọc theo từng "Batch" (`batch_size=200`) và đẩy xuống SQLite persistent queue (`Spool.db`).
3. Nếu RAM đầy, gói tin cũ/không quan trọng sẽ bị drop (Drop Policy - Fail-safe) để tránh crash hệ thống (OOM - Out of Memory).

Câu 16: (Khó) Hệ thống giải quyết vấn đề Sensor bị mất mạng tạm thời (Network Cut) như thế nào? Trả lời: Sensor có thiết kế "Soak/Backlog" cực kỳ chịu lỗi. Nếu luồng Upload gọi API thất bại, Sensor không vứt bỏ gói dữ liệu đó, mà đưa nó vào một **Backlog** (lưu ở RAM hoặc `spool.db`). Khi kết nối mạng phục hồi (`network_up = True`), luồng Upload sẽ tự động thực hiện "Drain Backlog" (rút cạn hàng đợi) bằng cách gửi dồn các lô dữ liệu cũ lên Controller.

Câu 17: (Bổ sung) Baseline Learning Mode trong Sensor hoạt động như thế nào? Trả lời: Baseline Manager học các hành vi "Bình thường" của mạng xung quanh trong một khoảng thời gian (Learning mode = True). Nó lưu cấu hình BSSID, chuẩn bảo mật, các kênh thường dùng và tính toán độ lệch chuẩn của sóng RSSI. Sau khi học xong, nếu sóng (RSSI) đột ngột mạnh lên hoặc chuẩn bảo mật bị hạ thấp, nó sẽ phát tín hiệu cảnh báo Deviation (Độ lệch chuẩn).

Phần 3: Thuật toán phát hiện tấn công, Exploit Chains & Geo-Location

Câu 18: (Nền tảng) Tấn công Evil Twin là gì? Tại sao nó nguy hiểm? Trả lời: Evil Twin là kẻ tấn công tạo ra một điểm phát sóng (AP) giả mạo có cùng tên (SSID) với mạng hợp pháp, thường phát tín hiệu mạnh hơn để lừa thiết bị người dùng kết nối vào. Từ đó, kẻ tấn công có thể nghe lén mật khẩu, đánh cắp cookie hoặc thực hiện tấn công Man-in-the-Middle (MitM).

Câu 19: Cuộc tấn công Deauth Flood (Từ chối dịch vụ WiFi) được hệ thống phát hiện dựa trên cơ chế nào? Trả lời: `DeauthFloodDetector` đếm số lượng gói tin Deauth/Disassoc hướng đến một Client hoặc Broadcast (ff:ff:ff:ff:ff:ff) trong một khoảng thời gian (sliding window). Nếu tốc độ (rate) vượt qua ngưỡng `threshold_per_sec` (ví dụ 10 gói/giây), hệ thống sẽ cảnh báo.

Câu 20: Làm sao hệ thống phát hiện ra điểm phát sóng giả mạo (Evil Twin)? Trả lời: Thuật toán `AdvancedEvilTwinDetector` dùng mô hình **chấm điểm trọng số (weighted scoring)**:

- Tìm các AP có cùng tên (SSID) nhưng khác BSSID (MAC).
- Cộng điểm nếu có dấu hiệu bất thường: Cường độ tín hiệu (RSSI) nhảy vọt (khoảng cách vật lý gần hơn), sai khác về OUI (nhà sản xuất), chuẩn bảo mật thay đổi, khoảng thời gian Beacon Jitter, và chênh lệch Information Elements (IEs).

Câu 21: Việc "Temporal confirmation" (Xác nhận theo thời gian) trong Evil Twin Detector có ý nghĩa gì? Trả lời: Tránh cảnh báo giả (False Positive) do nhiễu sóng hoặc roaming hợp lệ. Yêu cầu AP đáng ngờ phải tồn tại và duy trì điểm số ác ý trong một "cửa sổ thời gian" (`confirmation_window_seconds`) trước khi chính thức phát cảnh báo Critical.

Câu 22: Hệ thống làm thế nào để tránh việc cảnh báo liên tục một cuộc tấn công đang diễn ra (Alert Spam)? Trả lời: Sử dụng cơ chế Cooldown/Deduplication. Tại Sensor có `AlertManager` và tại thuật toán (như DoS) có lưu trạng thái `last_alert` . Một cuộc tấn công cùng loại vào cùng mục tiêu sẽ bị chặn (cooldown) trong ví dụ 60 giây trước khi có thể phát một cảnh báo mới.

Câu 23: Tại sao trong thuật toán phát hiện, cấu trúc dữ liệu Set (tập hợp) và Dictionary (bảng băm) lại được sử dụng nhiều (ví dụ O(1) lookups)? Trả lời: Vì dữ liệu mạng luân chuyển rất nhanh. Phải dùng Set/Dictionary để tìm kiếm (lookups) theo MAC/BSSID với độ phức tạp thời gian là O(1) (trong Python), đảm bảo hệ thống không bị nghẽn (bottleneck) so với việc phải chạy vòng lặp dùng List (mức O(N)). Gần đây dự án còn cập nhật thêm các hàm sắp xếp để tối ưu hóa hiệu suất (bolt-perf-sorting).

Câu 24: (Mới - Khó) KRACK (Key Reinstallation Attack) hoặc PMKID Attack được hệ thống phát hiện theo dấu hiệu kỹ thuật nào? Trả lời: Dù Tầng Data bị mã hóa WPA2/WPA3, các công cụ thu thập chứng chỉ bẻ khóa như MDK4, Wifite thường gửi các chuỗi gói tin EAPOL (4-way handshake) bất thường hoặc yêu cầu kết nối lại (Reassociation) liên tục. Engine phát hiện sẽ dò tìm tần suất xuất hiện các khung EAPOL/PMKID bất hợp lý trên một cặp địa chỉ BSSID-Client MAC để cảnh báo hành vi Brute-force/Bẻ khóa mật khẩu.

Câu 25: (Khó) Khái niệm "Exploit Chain" trong dự án là gì? Nó giúp phát hiện điều gì mà các thuật toán đơn lẻ không làm được? Trả lời: Exploit Chain (chuỗi khai thác) là việc kết hợp nhiều cảnh báo rời rạc lại với nhau để tìm ra một cuộc tấn công phức tạp. Ví dụ: Thuật toán đơn lẻ phát hiện (1) Một mạng Evil Twin và (2) Một cuộc Deauth Flood. Tuy nhiên, `ExploitChainAnalyzer` sẽ theo dõi và thấy kẻ tấn công vừa tạo Evil Twin, *ngay sau đó* bắn Deauth Flood để ép người dùng ngắt kết nối từ mạng gốc và văng sang mạng giả. Cảnh báo "Chain" sẽ có mức độ nghiêm trọng (Critical) cực cao.

Câu 26: (Khó) Cuộc tấn công KARMA/Pineapple (Karma Detector) hoạt động dựa trên cơ chế bắt gói tin nào? Trả lời: Client thường lưu tên các mạng WiFi cũ và liên tục phát ra `Probe Request` . Kẻ tấn công dùng WiFi Pineapple nghe lén Probe Request này và lập tức phát ra `Probe Response` giả mạo (trả lời "Có, tao là mạng X đây") bất chấp tên mạng là gì. Hệ thống phát hiện bằng cách tìm một AP (cùng BSSID) nhưng liên tục trả lời bằng nhiều SSID khác nhau trong thời gian ngắn.

Câu 27: Tính năng Geo-Mapping (Định vị địa lý) hoạt động dựa trên phương pháp toán học nào? Trả lời: Hệ thống sử dụng phương pháp **Trilateration (Phép đo ba góc/Định vị 3 điểm)**.

- Nó sử dụng "Log-distance path loss model" để chuyển đổi sóng vô tuyến (RSSI) thành khoảng cách ước tính (m).
- Khi có ít nhất 3 Sensors cùng thu được tín hiệu của kẻ tấn công, hệ thống áp dụng công thức đường tròn giao nhau (được làm mịn bằng bộ lọc Kalman để khử nhiễu) để tính ra tọa độ x,y của thiết bị mục tiêu.

Phần 4: Backend Controller, API & Authentication (RBAC)

Câu 28: Tại sao API Server của Controller (Flask) không nhận trực tiếp Raw PCAP mà lại nhận JSON Telemetry? Trả lời: Để tối ưu băng thông và bảo mật. Gói tin PCAP rất lớn và chứa nội dung dữ liệu (payload). JSON Telemetry chỉ chứa các siêu dữ liệu (metadata) đã được trích xuất (MAC, RSSI, loại Frame, mã hóa) và kích thước rất nhỏ, phù hợp truyền tải qua Internet.

Câu 29: An toàn thông tin giữa Sensor và Controller được đảm bảo như thế nào? Trả lời: Controller sử dụng mTLS (hoặc TLS/HTTPS cơ bản) để mã hóa đường truyền. Đồng thời, xác thực các Request tải dữ liệu từ Sensor bằng API Key và HMAC-SHA256 Signature, nhằm đảm bảo dữ liệu không bị giả mạo trên đường truyền.

Câu 30: Middleware trong Flask API đóng vai trò gì trong hệ thống này? Trả lời: Middleware (như `ObservabilityMiddleware`, `TrustedProxyMiddleware`) chạy trước khi request vào logic chính. Nó giúp đếm số lượng request (tạo metrics cho Prometheus), kiểm tra giới hạn tốc độ (Rate Limiting), và phân tích chính xác IP thật của Sensor nếu đi qua Nginx/Proxy.

Câu 31: Quản lý quyền truy cập RBAC (Role-Based Access Control) được triển khai như thế nào trong Controller? Trả lời: Tại `controller/api/auth.py`, hệ thống phân quyền rõ ràng theo Token:

- **Role.SENSOR:** Chỉ có quyền gọi lệnh POST để gửi telemetry và heartbeats lên máy chủ.
- **Role.ADMIN / DASHBOARD:** Được phép gọi các API đọc/sửa danh sách cảnh báo, quản lý sensors, và tải report. Token này được cấp với danh sách quyền hạn dạng Enum (`list(Permission)`) giới hạn rủi ro leo thang đặc quyền.

Câu 32: (Mới - Database) Tại sao lưu trữ log/telemetry lại khuyên dùng Elasticsearch/Time-Series DB (TSDB) thay vì PostgreSQL truyền thống? Trả lời: PostgreSQL (RDBMS) rất tốt cho cấu hình Sensor, Danh mục mạng, RBAC. Tuy nhiên, với Telemetry sóng WiFi, một phút có thể sinh ra hàng nghìn bản ghi. TSDB (như Elasticsearch hoặc InfluxDB) tối ưu hóa kiến trúc file (Append-only) để ghi tốc độ cao, hỗ trợ Index thời gian, tự động nén data và dễ dàng query thống kê theo biểu đồ (vd: đếm số lượng probe request mỗi phút).

Phần 5: Machine Learning (Phân tích bất thường) & Risk Scoring

Câu 33: (Nền tảng) Risk Scoring (Chấm điểm rủi ro) hoạt động ra sao và nó có tính linh hoạt thế nào? Trả lời: Module `algos/risk.py` chấm điểm từ 0-100 cho một BSSID dựa trên các đặc trưng như: Mã hóa (Encryption), Tín hiệu (Signal), Lịch sử xuất hiện... Nó dùng hệ thống **Trọng số (ScoringWeights)** (Ví dụ: Encryption chiếm 50%, Signal 10%). Điều này cho phép quản trị viên "căn chỉnh" (calibrate) lại trọng số để phù hợp hơn với từng môi trường cụ thể (Ví dụ: Giảm trọng số Signal nếu môi trường nhiễu cao).

Câu 34: Mô hình Machine Learning trong dự án sử dụng kiến trúc mạng nơ-ron nào và tại sao? Trả lời: Sử dụng **Autoencoder** (Mạng nơ-ron tự mã hóa - PyTorch). Autoencoder học cách "nén" và "giải nén" các dữ liệu mạng bình thường (baseline). Với mạng bình thường, lỗi giải nén (Reconstruction Error - MSE Loss) sẽ thấp.

Câu 35: Làm sao Autoencoder phát hiện được sự bất thường (Anomaly)? Trả lời: Khi có một vector đặc trưng mạng chứa dấu hiệu tấn công (chưa từng xuất hiện trong lúc học), Autoencoder sẽ không thể giải nén chính xác, dẫn đến sai số (MSE Loss) vượt qua một ngưỡng (Threshold) đã định. Khi đó, nó kết luận đó là Anomaly.

Câu 36: ML Autoencoder ở đây là học có giám sát (Supervised) hay không giám sát (Unsupervised)? Lý do chọn? Trả lời: Là **Học không giám sát (Unsupervised)**. Lý do là trong

thực tế, các cuộc tấn công Zero-day hoặc hành vi biến đổi rất khó có đủ nhãn (labels) để huấn luyện. Unsupervised learning chỉ cần học hành vi "bình thường" để phát hiện bất cứ thứ gì "bất thường".

Câu 37: Hệ thống Hybrid (Lai ghép) kết hợp giữa Risk Scoring bằng tay (Heuristics) và Autoencoder ML như thế nào? Trả lời: Đây là điểm sáng của WIDS Lai ghép:

1. Gói tin đi qua Rule-based/Heuristics (VD: `AdvancedEvilTwinDetector`, `RiskScorer`) để tính ra một điểm rủi ro cơ sở dựa trên chuyên môn con người.
2. Gói tin tiếp tục đi qua ML Autoencoder.
3. Nếu ML nhận diện đây là hành vi rất bất thường (Reconstruction loss cao), nó sẽ tính ra một điểm `deviation_score` và cộng gộp / kích (boost) điểm số rủi ro ban đầu lên cao hơn, giúp bù đắp những lỗ hổng mà luật tĩnh (static rules) của con người chưa bao phủ tới.

Phần 6: Vận hành, Dashboard, Thực tế & Bảo mật dự án

Câu 38: Giao diện Dashboard được xây dựng bằng công nghệ gì và nó có tính năng Real-time ra sao? Trả lời: Dashboard dùng **Plotly Dash** (Python), cho phép tạo giao diện Multi-page dạng Web. Tính Real-time đạt được thông qua component `dcc.Interval`, cứ mỗi X giây giao diện sẽ tự động gửi request đến Controller API để kéo dữ liệu mới và cập nhật biểu đồ / bản đồ mà không cần người dùng f5 lại trang web. Giao diện dùng `dbc.themes.DARKLY` để tăng độ tương phản hiển thị cảnh báo.

Câu 39: (Mới - Vận hành) Màn hình cấu hình (TUI - Terminal User Interface) của Sensor dùng để làm gì? Trả lời: Trong các cập nhật mới (`sensor/tui/setup_wizard.py`), TUI là trình hướng dẫn (wizard) chạy trên giao diện dòng lệnh. Nó cho phép quản trị viên cài đặt nhanh Lab mode, tự động cấp quyền Sensor (generate tokens), dò tìm card WiFi (Wireless Inventory), và cấu hình các tham số Drop Policy, Dwell Time mà không cần sửa file JSON tĩnh bằng tay phức tạp.

Câu 40: "Lab Mode" và "Production Mode" khác nhau thế nào trong dự án này? Trả lời:

- **Lab Mode:** Dùng SQLite, Docker Compose nhẹ nhàng, cấu hình bảo mật lỏng lẻo (http), Mock Interface (để replay file Wardrive JSON/PCAP), hướng tới mục đích demo, học tập.
- **Production Mode:** Dùng PostgreSQL, có mTLS, yêu cầu cấu hình bí mật (.env chặt chẽ), chạy bằng Nginx/Gunicorn, hệ thống giám sát Observability (Prometheus/Grafana) thực thụ.

Câu 41: Hệ thống Metric bằng Prometheus hoạt động ra sao trong WIDS? Trả lời: Bằng việc import `prometheus_client` (`common/observability/metrics.py`), hệ thống tạo ra các Endpoint xuất dữ liệu hiệu năng dạng Counter/Gauge (như Số luồng kết nối, RAM sử dụng, Lưu lượng frames bắt được). Quản trị viên chỉ cần tích hợp Grafana để đọc các chỉ số sức khoẻ này theo thời gian thực.

Câu 42: Trong hệ thống cấu hình (`sensor/config.py`), dự án sử dụng mô hình "Fail-Fast Secret" là gì và tại sao nó quan trọng? Trả lời: Fail-Fast Secret (Lỗi nhanh) là nguyên tắc thiết kế yêu cầu ứng dụng phải **chết/dừng khởi động ngay lập tức** nếu các biến môi trường nhạy cảm (như SENSOR_HMAC_SECRET, API_KEY) bị thiếu, độ dài quá ngắn, hoặc quá yếu. Thay vì để hệ thống chạy ngầm với cấu hình lỏng lẻo và bị hack sau này, việc "crash" sớm buộc quản trị viên phải cấu hình đúng chuẩn bảo mật trước khi hệ thống lên sóng.

Câu 43: Dự án tuân thủ quyền riêng tư (Privacy) của người dùng mạng WiFi như thế nào? Trả lời: Hệ thống cung cấp module `common/privacy.py`. Các địa chỉ MAC (vd MAC của điện thoại) sẽ được mã hóa băm một chiều (Hash) kết hợp với một `Salt` ngẫu nhiên trước khi gửi về trung tâm hoặc lưu trữ. Do đó, hệ thống không lưu trữ danh tính thật (MAC Gốc) của thiết bị cá nhân, tuân thủ các quy định Data Privacy.

Câu 44: Nếu một kẻ tấn công gửi ngập lụt (flood) các cảnh báo giả mạo lên API của Controller nhằm làm sập Database, em xử lý tình huống đó ở Backend ra sao? Trả lời: Dự án sử dụng kết hợp nhiều lớp phòng thủ:

1. **Rate Limiting:** Middleware giới hạn số lượng request API/phút (ví dụ Flask-Limiter).
2. **Authentication:** Bắt buộc có API Token hợp lệ và kiểm tra HMAC signature cho mỗi payload. Nếu sai, rớt ngay ở vòng ngoài (401/403).
3. **Queue / Message Broker:** API không ghi trực tiếp vào Database PostgreSQL, mà sẽ đưa dữ liệu vào Redis Queue/Celery worker để xử lý bất đồng bộ, giúp DB không bị khóa nghẽn (locking) khi có luồng dữ liệu khổng lồ đẩy tới.

Câu 45: Khó khăn lớn nhất trong việc kiểm thử (Testing) module Sensor thu thập gói tin không dây (Wi-Fi 802.11) là gì và dự án giải quyết như thế nào? Trả lời: Khó khăn lớn nhất là việc bắt gói tin thực tế (Live Capture) đòi hỏi quyền root (sudo) và phần cứng card mạng đặc thù hỗ trợ Monitor Mode. Dự án giải quyết bằng cách áp dụng **Dependency Injection (Mocking)**: Thay vì dùng card thật, bài test dùng `MockCaptureDriver` để tự sinh ra các gói dữ liệu ảo, hoặc dùng `PcapCaptureDriver` để đọc và "replay" (phát lại) dữ liệu từ các file `.pcap` đã ghi âm sẵn. Điều này giúp hệ thống test (CI/CD) có thể chạy ổn định trên mọi máy chủ mà không cần phần cứng WiFi vật lý.

Câu 46: Gói tin khi vận chuyển từ Sensor lên Controller (`sensor/transport.py`) được cấu trúc và bảo vệ như thế nào? Trả lời: Để đảm bảo an toàn và tối ưu, gói tin được đi qua 3 bước:

1. **JSON Payload:** Dữ liệu chuẩn hóa.
2. **Nén (Compression):** Nén bằng thuật toán `gzip` để tiết kiệm băng thông khi truyền qua mạng (nhất là mạng di động/4G cho Sensor xa).
3. **Chữ ký điện tử (Signature):** Máy chủ tính mã băm HMAC-SHA256 trên nội dung gói tin nén cộng với `X-Timestamp` để chống lại tấn công Replay (phát lại) và giả mạo dữ liệu.

Câu 47: Đảm bảo tính sẵn sàng cao (High Availability), hệ thống Sentinel NetLab thiết kế cơ chế tự phục hồi (Auto-recovery) như thế nào khi gặp sự cố? Trả lời: Hệ thống có thiết kế chống chịu lỗi (fault-tolerant) trên 4 cấp độ:

1. **Cấp độ Process:** Sensor dùng `Systemd` (Restart=on-failure), còn Controller dùng `Docker Compose` (restart: unless-stopped) kết hợp Healthchecks.
2. **Cấp độ Application:** Mô hình "Fail-fast". Liên tục kiểm tra trạng thái Monitor mode. Nếu phát hiện card mạng bị treo (không có gói mới trong 30s), tiến trình tự crash để ép Systemd khởi động lại sạch sẽ.
3. **Cấp độ Hardware:** Có module `USBWatchdog` giám sát card mạng rời. Tự khôi phục driver mạng (modprobe) khi USB WiFi bị ngắt/lỗi.
4. **Cấp độ Data:** Sử dụng Persistent Queue (SQLite Spool) lưu tạm dữ liệu lúc rớt mạng, tự động phục hồi "inflight packets" và đẩy lại (drain backlog) khi mạng có lại.

Câu 48: Nếu nâng cấp hệ thống trong tương lai, em sẽ cải thiện điều gì? Trả lời: (Gợi ý trả lời)

1. Thêm hỗ trợ phân tích khung bảo mật WPA3 PMF (Protected Management Frames).
2. Triển khai kiến trúc xử lý Stream lớn (Apache Kafka/Kinesis) thay cho REST API nếu mô hình tăng lên hàng nghìn cảm biến.
3. Chạy thuật toán AI tại biên (Edge AI) thông qua phần cứng TPU.
4. Triển khai các kịch bản Active Defense (Cách ly kẻ tấn công hoàn toàn).

Phần 7: Dữ liệu Thực nghiệm & Đánh giá Báo cáo Đồ án (Capstone Project)

Câu 49: Mặc dù đã có các cơ chế bảo mật mạnh mẽ như WPA2-AES hay WPA3, tại sao mạng WiFi vẫn tồn tại lỗ hổng cho các cuộc tấn công nhắm vào tính sẵn sàng (Deauth Flood)? Trả lời: Điểm yếu chí mạng của chuẩn 802.11 b/g/n (và thường cả ac) nằm ở việc thiếu cơ chế xác thực gốc cho "Management Frames" (Khung quản lý). Những khung như Deauthentication, Probe Request, Beacon được truyền dưới dạng bản rõ (cleartext). Do đó, kể cả Data Plane có mã hóa WPA3 mạnh đến đâu, kẻ tấn công vẫn có thể giả mạo (MAC Spoofing) các khung quản lý này để gây ngắt kết nối.

Câu 50: Tại sao WIDS lại vượt trội hơn Firewall truyền thống trong việc phát hiện tấn công WiFi? Trả lời: Firewall truyền thống thường hoạt động ở Tầng 3 (Network Layer - IP) trở lên và hoàn toàn "mù" trước các bất thường xảy ra ở Tầng 2 (Data Link Layer) của môi trường sóng vô tuyến (Radio). WIDS bắt gói tin trực tiếp ở Tầng 2 nên có thể phát hiện các hành vi dò tìm (Probe), nhân bản BSSID (Evil Twin) hay ngắt kết nối (Deauth) mà Firewall không thể nhìn thấy.

Câu 51: Trong quá trình đánh giá (Benchmark), các chỉ số Precision, Recall và F1-Score mang ý nghĩa gì? Trả lời:

- **Precision:** Tỉ lệ phát hiện đúng (Bao nhiêu % trong số các cảnh báo hệ thống phát ra là tấn công thật sự).
- **Recall:** Tỉ lệ bắt trúng (Hệ thống không bỏ lọt bao nhiêu % trong tổng số các cuộc tấn công đã xảy ra).
- **F1-Score:** Là trung bình điều hòa của Precision và Recall, đặc biệt quan trọng trong tập dữ liệu mất cân bằng (imbalanced) khi số lượng gói tin "Benign" (Bình thường) nhiều gấp hàng triệu lần số lượng gói tin tấn công.

Câu 52: (Quan trọng) Chỉ số FPR (False Positive Rate) là gì và tại sao "Very Low FPR" (Tỉ lệ báo động giả rất thấp) là yếu tố sống còn của một WIDS? Trả lời: FPR là tỉ lệ hệ thống nhận diện nhầm một hành vi hợp lệ thành một cuộc tấn công. Nếu FPR cao, hệ thống sẽ gây ra "Alert Fatigue" (Hội chứng kiệt sức vì báo động) khiến người quản trị bỏ qua hoặc tắt luôn WIDS. Kiến trúc Hybrid của Sentinel đạt FPR rất thấp nhờ việc "xác thực chéo" (cross-referencing) giữa luật tĩnh và các chỉ số vật lý như tín hiệu (RSSI) và phần cứng (OUI).

Câu 53: Bộ dữ liệu (Test Dataset) để kiểm thử hệ thống được xây dựng như thế nào? Trả lời: Để đảm bảo tính khách quan và kiểm thử trên mọi môi trường độc lập, dự án dùng:

- **Golden/Synthetic PCAP:** Các file ghi lại gói tin chứa kịch bản tấn công kinh điển (Deauth, Evil Twin) để mô phỏng.
- **Sample Labeled Dataset:** Tập dữ liệu có dán nhãn (Benign, rogue_ap, evil_twin, deauth_flood) trích xuất trực tiếp từ các bài test chạy trên Lab để đo đạc độ chính xác của ML và Rules.

Câu 54: So sánh hiệu năng của 3 phương pháp tiếp cận: Rule-only, ML-only và Hybrid (Lai ghép)? Trả lời: Dựa trên kết quả thực nghiệm:

- **Rule-only:** Precision cao, bắt rất nhanh các dạng tấn công đã biết (MDK4 Deauth) nhưng Recall trung bình vì dễ bỏ lọt các biến thể tấn công mới.
- **ML-only:** Recall cao vì tìm ra được bất thường Zero-day, nhưng Precision thấp và FPR (báo động giả) cao vì mọi biến động nhiễu sóng hợp lệ đều bị coi là bất thường.
- **Hybrid (Đề xuất):** Giao thoa điểm mạnh. Tốc độ của Rule cộng với chiều sâu phân tích của ML (cùng kiểm tra dấu vân tay vật lý RSSI/BSSID) đem lại F1-Score rất cao và FPR cực thấp.

Câu 55: Yếu tố MTTD (Mean Time To Detect) trong dự án đạt được là bao nhiêu và được tối ưu nhờ thiết kế nào? Trả lời: Hệ thống đạt MTTD rất thấp (độ trễ dưới 1 giây - sub-second thresholds). Điều này có được là nhờ kiến trúc Edge Computing: Thay vì phải đóng gói toàn bộ Raw PCAP gửi về Backend gây nghẽn cổ chai mạng, việc phân tích, trích xuất đặc trưng và đối chiếu luật

diễn ra ngay tại Raspberry Pi (Sensor), sau đó chỉ tải một khối lượng nhỏ metadata (JSON) về Controller để hiển thị cảnh báo ngay lập tức.